

GRA4158 Marketing - Analysis of Experiments, Portfolio 4

Vilijam Cekov, Luiz Felipe De Souza Simao

Table of contents

1	Import Data	2
2	A simple pre-post analysis of only the treated store for each product controlling for relevant covariates	3
3	A simple two-way fixed-effect panel model for each product controlling for relevant covariates	4
4	Simple DiD, SC, and SDID analyses for each product	6
4.1	Check for parallel trends assumption	6
4.1.1	Visual inspection of pre-treatment trends	6
4.1.2	Statistical test for parallel trends: Pre-treatment interaction model	8
4.2	Evaluating the treatment effect using DiD, SC, and SDID methods for each product	9
4.2.1	Product 1: p1sales	9
4.2.2	Product 2: p2sales	11
4.2.3	SC diagnostics: Pre/Post RMSPE and donor weights	13
5	Expansions to the analysis	15
5.1	Pooled DiD with covariates	16
5.1.1	Product 1: Pooled DiD with covariates	16
5.1.2	Product 2: Pooled DiD with covariates	16
5.2	Synthetic DiD regressing out the covariates	17
5.2.1	Product 1: p1sales	17
5.2.2	Product 2: p2sales	18
6	Summary of Findings	20
6.1	Conclusions	20
6.1.1	Pre-Post Analysis	20

6.1.2	Two-Way Fixed Effects (TWFE) Analysis	21
6.1.3	Difference-in-Differences (DID) Analysis	21
6.1.4	Pooled DiD with Covariates Analysis	22
6.1.5	Synthetic Control (SC) Analysis	22
6.1.6	Synthetic Difference-in-Differences (SDID) and Adjusted SDID Analysis	23
6.2	Final decision:	24
6.2.1	Recommendations to Management	24
6.2.2	Effectiveness and Expected Sales Uplift	24
6.2.3	Potential Challenges in the Data	24
6.2.4	Ideas to Overcome These Challenges	25

1 Import Data

```
df <- read_delim("Store_data_2026_taks4.csv", delim = ";")

# Create necessary dummy variables
df <- df %>%
  mutate(
    # 'post_treatment' is 1 for the weeks after the marketing strategy started
    post_treatment = ifelse(Weekind >= 78, 1, 0),
    # 'treated' is the actual interaction / DiD term
    # (1 ONLY for Store 109 AND Weekind >= 78)
    treated = ifelse(storeNum == 109 & Weekind >= 78, 1, 0),
    # Variable identifying the treated store (regardless of time)
    is_treated_store = ifelse(storeNum == 109, 1, 0)
  )

# Display the structure of the data to verify column names
str(df)
```

```
tibble [2,184 x 18] (S3: tbl_df/tbl/data.frame)
 $ storeNum      : num [1:2184] 101 101 101 101 101 101 101 101 101 101 ...
 $ Year          : num [1:2184] 2024 2024 2024 2024 2024 2024 ...
 $ Week          : num [1:2184] 1 2 3 4 5 6 7 8 9 10 ...
 $ Date          : Date[1:2184], format: "2024-01-08" "2024-01-15" ...
 $ Weekind       : num [1:2184] 1 2 3 4 5 6 7 8 9 10 ...
 $ p1sales       : num [1:2184] 457 500 456 450 454 449 453 450 498 458 ...
 $ p2sales       : num [1:2184] 588 707 591 591 590 593 710 591 594 592 ...
 $ p1price       : num [1:2184] 2.49 2.99 2.19 2.29 2.19 2.79 2.49 2.99 2.79 2.79 ...
 $ p2price       : num [1:2184] 2.99 3.19 2.99 3.19 3.19 2.59 3.19 3.19 2.99 3.19 ...
```

```

$ p1prom      : num [1:2184] 0 1 0 0 0 0 0 0 1 0 ...
$ p2prom      : num [1:2184] 0 1 0 0 0 0 1 0 0 0 ...
$ compind     : num [1:2184] 0.123 0.123 0.123 0.123 0.123 0.123 0.123 0.123 0.123 0.123 ...
$ storesize   : num [1:2184] 143 143 143 143 143 143 143 143 143 143 ...
$ city        : chr [1:2184] "OSLO" "OSLO" "OSLO" "OSLO" ...
$ citysize    : num [1:2184] 730000 730000 730000 730000 730000 730000 730000 730000 730000 730000 ...
$ post_treatment : num [1:2184] 0 0 0 0 0 0 0 0 0 0 ...
$ treated     : num [1:2184] 0 0 0 0 0 0 0 0 0 0 ...
$ is_treated_store: num [1:2184] 0 0 0 0 0 0 0 0 0 0 ...

```

```

# Check the first few rows
head(df)

```

```

# A tibble: 6 x 18
  storeNum Year Week Date      Weekind p1sales p2sales p1price p2price p1prom
  <dbl> <dbl> <dbl> <date>      <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
1     101  2024     1 2024-01-08         1     457     588     2.49     2.99     0
2     101  2024     2 2024-01-15         2     500     707     2.99     3.19     1
3     101  2024     3 2024-01-22         3     456     591     2.19     2.99     0
4     101  2024     4 2024-01-29         4     450     591     2.29     3.19     0
5     101  2024     5 2024-02-05         5     454     590     2.19     3.19     0
6     101  2024     6 2024-02-12         6     449     593     2.79     2.59     0
# i 8 more variables: p2prom <dbl>, compind <dbl>, storesize <dbl>, city <chr>,
#   citysize <dbl>, post_treatment <dbl>, treated <dbl>, is_treated_store <dbl>

```

2 A simple pre-post analysis of only the treated store for each product controlling for relevant covariates

The relevant covariates chosen are price and promotion for each product, as they are likely to influence sales. Time-invariant covariates such as store size and city size are excluded from this analysis since these stable factors are automatically controlled without requiring additional covariates. For this method to output an appropriate causal estimate, we would need to assume that no other external factors influenced sales at Store 109 between the pre- and post-treatment weeks. (This is a strong and unlikely assumption for this dataset, which is why we expect that more advanced methods like Two-Way Fixed Effects or Synthetic Control are generally more appropriate for establishing causality)

```

# Filter data to only include Store 109
df_109 <- df %>% filter(storeNum == 109)

```

```
# --- Product 1: Pre-Post Model ---
pre_post_p1 <- lm(p1sales ~ post_treatment + p1price + p1prom, data = df_109)
coeftest(pre_post_p1)["post_treatment", , drop = FALSE]
```

	Estimate	Std. Error	t value	Pr(> t)
post_treatment	33.68939	1.710744	19.69283	3.444753e-36

```
# --- Product 2: Pre-Post Model ---
pre_post_p2 <- lm(p2sales ~ post_treatment + p2price + p2prom, data = df_109)
coeftest(pre_post_p2)["post_treatment", , drop = FALSE]
```

	Estimate	Std. Error	t value	Pr(> t)
post_treatment	-29.71007	1.870237	-15.88572	4.193528e-29

3 A simple two-way fixed-effect panel model for each product controlling for relevant covariates

Store fixed effects will automatically absorb (control for) all time-invariant store characteristics (like storesize and city). Time fixed effects will absorb aggregate shocks that affect all stores in a given week. By accounting for these factors, the TWFE model addresses the primary endogeneity problem of unobserved effects that are either subject-invariant or time-invariant. While the simple pre-post model assumes the counterfactual outcome remains constant over time, the TWFE model (within a Difference-in-Differences framework) relaxes this by assuming that treatment and control groups would follow parallel trends in the absence of treatment. Where this is assumed to produce smaller standard errors and tighter confidence intervals and allowing for a more granular analysis of heterogeneity on the individual and time dimensions rather than just group-level averages in contrast to pre-post analysis, the introduction of the parallel trends assumption also means that if the treated store had a unique sales trajectory that was not shared by any control stores, the TWFE estimate could be biased.

```
# Convert the data frame to a panel data frame for plm
pdf <- pdata.frame(df, index = c("storeNum", "Weekind"))

# --- Product 1: TWFE Model ---
twfe_p1 <- plm(p1sales ~ treated + p1price + p1prom,
              data = pdf,
              model = "within",
              effect = "twoways")
coeftest(twfe_p1)["treated", , drop = FALSE]
```

	Estimate	Std. Error	t value	Pr(> t)
treated	29.97912	2.566882	11.6792	1.466622e-30

```
# --- Product 2: TWFE Model ---
twfe_p2 <- plm(p2sales ~ treated + p2price + p2prom,
              data = pdf,
              model = "within",
              effect = "twoways")
coeftest(twfe_p2)["treated", , drop = FALSE]
```

	Estimate	Std. Error	t value	Pr(> t)
treated	-34.29122	3.071355	-11.16485	3.833684e-28

Correcting for autocorrelation and heteroskedasticity

```
# --- Product 1 TWFE with Clustered Standard Errors ---
coeftest(twfe_p1, vcov = vcovHC(twfe_p1, type = "HC1", cluster = "group"))
```

t test of coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
treated	29.9791	4.0348	7.4301	1.582e-13	***
p1price	-2.4746	0.8446	-2.9299	0.003428	**
p1prom	37.5902	1.3932	26.9812	< 2.2e-16	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
# --- Product 2 TWFE with Clustered Standard Errors ---
coeftest(twfe_p2, vcov = vcovHC(twfe_p2, type = "HC1", cluster = "group"))
```

t test of coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
treated	-34.29122	4.45448	-7.6981	2.129e-14	***
p2price	-2.95105	0.81652	-3.6142	0.0003085	***
p2prom	89.42444	4.30234	20.7851	< 2.2e-16	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

4 Simple DiD, SC, and SDID analyses for each product

4.1 Check for parallel trends assumption

4.1.1 Visual inspection of pre-treatment trends

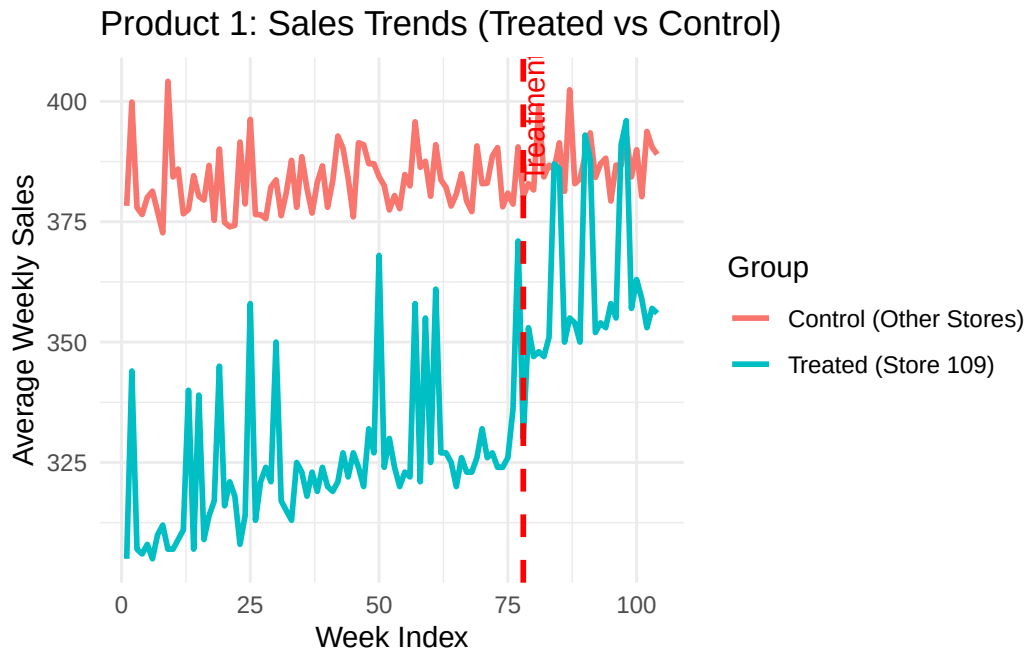
Ideally, the treated store's sales should follow a similar trajectory to the control stores in the pre-treatment period. If the treated store had a unique sales pattern that was not shared by any control stores, this would violate the parallel trends assumption and could bias our estimates. By plotting the average sales of the treated store against the average sales of the control stores over time, we can visually assess whether they exhibit similar trends before the treatment starts (Weekind < 78). If they do, this supports the validity of our DiD and SDID analyses; if not, we may need to consider alternative methods or be cautious in interpreting our results.

```
# --- Product 1 Plot ---
# Aggregate average sales per week for treated vs control stores
trend_data_p1 <- df %>%
  group_by(Weekind, is_treated_store) %>%
  summarise(mean_sales = mean(plsales, na.rm = TRUE),
            .groups = 'drop') %>%
  mutate(Group = ifelse(is_treated_store == 1,
                        "Treated (Store 109)",
                        "Control (Other Stores)"))

plot_p1 <- ggplot(trend_data_p1,
                 aes(x = Weekind, y = mean_sales, color = Group)) +
  geom_line(linewidth = 1) +
  geom_vline(xintercept = 78,
            linetype = "dashed",
            color = "red",
            linewidth = 1) +
  annotate("text",
         x = 76,
         y = max(trend_data_p1$mean_sales),
         label = "Treatment Start",
         angle = 90,
         vjust = 1.5,
         color = "red") +
  labs(title = "Product 1: Sales Trends (Treated vs Control)",
       x = "Week Index",
       y = "Average Weekly Sales") +
```

```
theme_minimal()

print(plot_p1)
```



```
# --- Product 2 Plot ---
trend_data_p2 <- df %>%
  group_by(Weekend, is_treated_store) %>%
  summarise(mean_sales = mean(p2sales, na.rm = TRUE), .groups = 'drop') %>%
  mutate(Group = ifelse(is_treated_store == 1,
                        "Treated (Store 109)",
                        "Control (Other Stores)"))

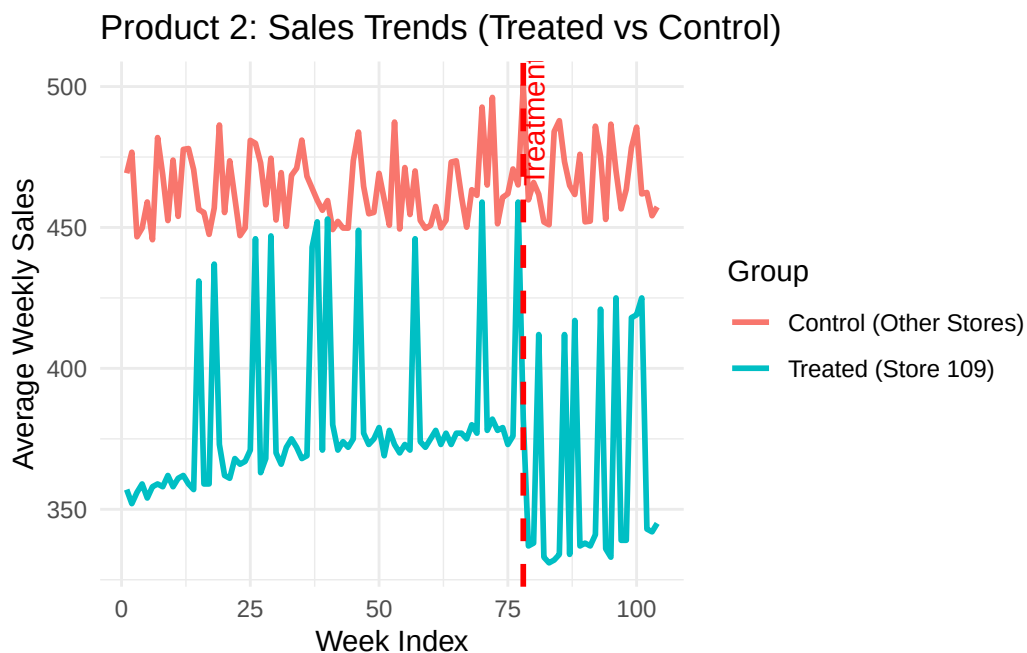
plot_p2 <- ggplot(trend_data_p2, aes(x = Weekend,
                                     y = mean_sales,
                                     color = Group)) +
  geom_line(linewidth = 1) +
  geom_vline(xintercept = 78,
             linetype = "dashed",
             color = "red",
             linewidth = 1) +
  annotate("text",
         x = 76,
```

```

        y = max(trend_data_p2$mean_sales),
        label = "Treatment Start",
        angle = 90,
        vjust = 1.5,
        color = "red") +
  labs(title = "Product 2: Sales Trends (Treated vs Control)",
        x = "Week Index",
        y = "Average Weekly Sales") +
  theme_minimal()

print(plot_p2)

```



4.1.2 Statistical test for parallel trends: Pre-treatment interaction model

Additionally to doing the visual inspection, we can also run a formal statistical test for parallel trends by regressing sales on the store dummy, a linear time trend (Weekind), and their interaction, but only using the pre-treatment data (Weeks 1 to 77). If the coefficient on the interaction term is statistically significant, it would suggest that the treated store had a different trend compared to the control stores even before the treatment started, which would violate the parallel trends assumption and potentially bias our DiD and SDID estimates.


```
# Filter the data to ONLY include the pre-treatment period (Weeks 1 to 77)
df_pre <- df %>% filter(Weekind < 78)
```

```
# --- Product 1: Parallel Trends Test ---
pt_test_p1 <- lm(p1sales ~ is_treated_store * Weekind, data = df_pre)
coeftest(pt_test_p1)["is_treated_store:Weekind", , drop = FALSE]
```

	Estimate	Std. Error	t value	Pr(> t)
is_treated_store:Weekind	0.2435446	0.3551978	0.6856591	0.4930266

```
# --- Product 2: Parallel Trends Test ---
pt_test_p2 <- lm(p2sales ~ is_treated_store * Weekind, data = df_pre)
coeftest(pt_test_p2)["is_treated_store:Weekind", , drop = FALSE]
```

	Estimate	Std. Error	t value	Pr(> t)
is_treated_store:Weekind	0.3467782	0.5785286	0.5994142	0.5489809

Findings: For both products, the visual interpretation leads to believe that there is a slight positive trend in the treated store's sales compared to the control stores in the pre-treatment period, which could potentially violate the parallel trends assumption. However, the formal statistical test for parallel trends (the interaction term between the treated store dummy and the linear time trend [is_treated_store:Weekind]) is not statistically significant for either product ($p > 0.05$). This suggests that while there may be some visual differences in trends, they are not statistically significant enough to conclusively reject the parallel trends assumption possibly due to limited sample size or high variability. Therefore, we can proceed with our DiD and SDID analyses, but we should be cautious in interpreting the results given the potential for some underlying differences in pre-treatment trends.

4.2 Evaluating the treatment effect using DiD, SC, and SDID methods for each product

4.2.1 Product 1: p1sales

```
# Ensure data frame is standard (synthdid prefers plain data.frames over tibbles)
df <- as.data.frame(df)

# Setup panel matrices for Product 1
setup_p1 <- panel.matrices(
```

```

df,
unit = "storeNum",
time = "Weekind",
outcome = "p1sales",
treatment = "treated"
)

# Estimate the treatment effects for Product 1
sdid_est_p1 <- synthdid_estimate(setup_p1$Y, setup_p1$N0, setup_p1$T0)
did_est_p1 <- did_estimate(setup_p1$Y, setup_p1$N0, setup_p1$T0)
sc_est_p1 <- sc_estimate(setup_p1$Y, setup_p1$N0, setup_p1$T0)
estimates_p1 = list(did_est_p1, sc_est_p1, sdid_est_p1)

# Print the estimates
names(estimates_p1) = c('Diff-in-Diff', 'Synthetic Control', 'Synthetic Diff-in-Diff')
print(unlist(estimates_p1))

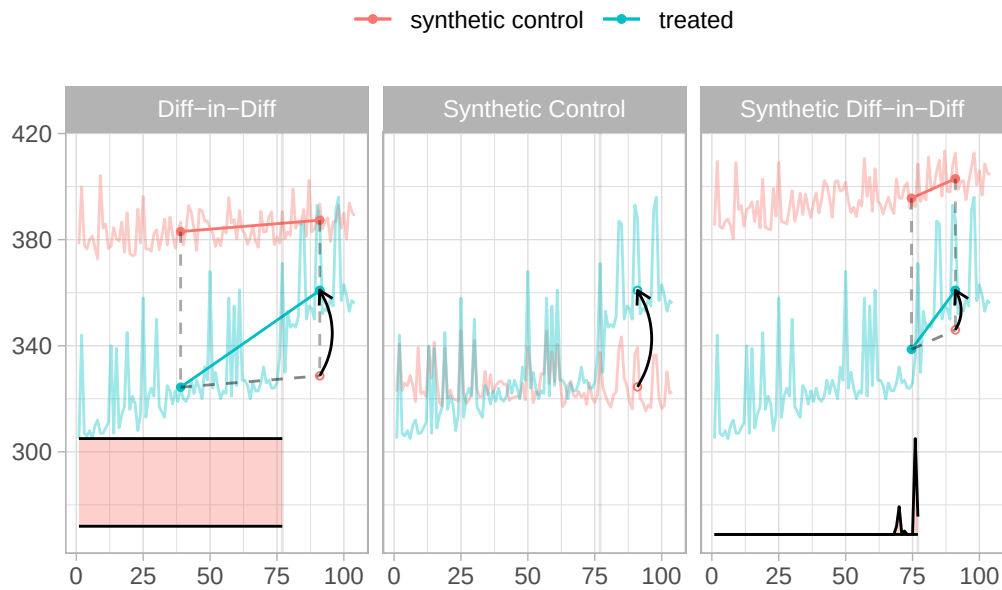
```

Diff-in-Diff	Synthetic Control	Synthetic Diff-in-Diff
32.21775	36.37194	14.88871

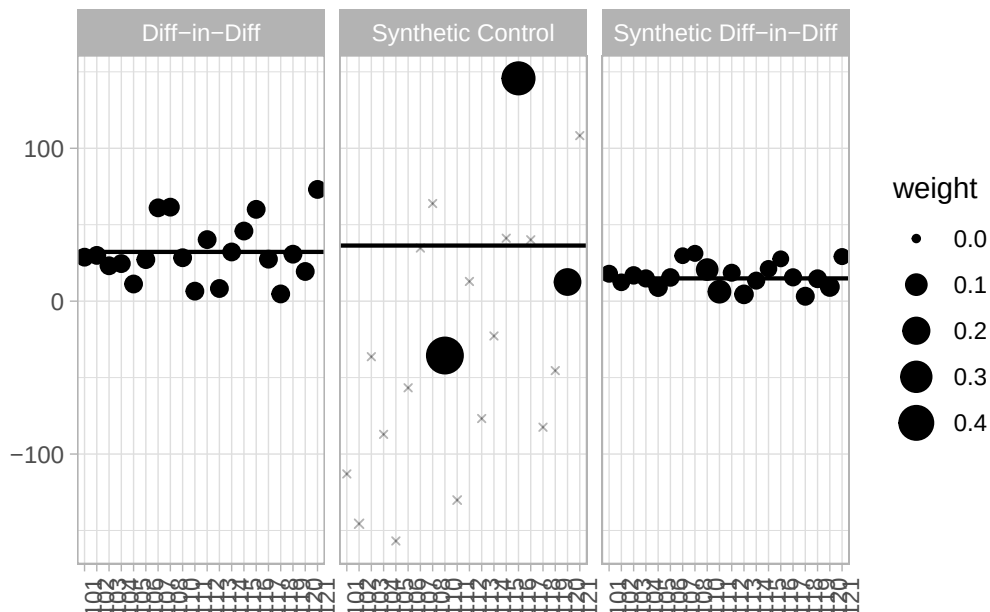
```

# Plot 1: Trends of treated vs. synthetic control
synthdid_plot(estimates_p1, facet.vertical = FALSE)

```



```
# Plot 2: See which control stores received the highest weights
synthdid_units_plot(estimates_p1)
```



4.2.2 Product 2: p2sales

The same steps as for Product 1 are repeated for Product 2.

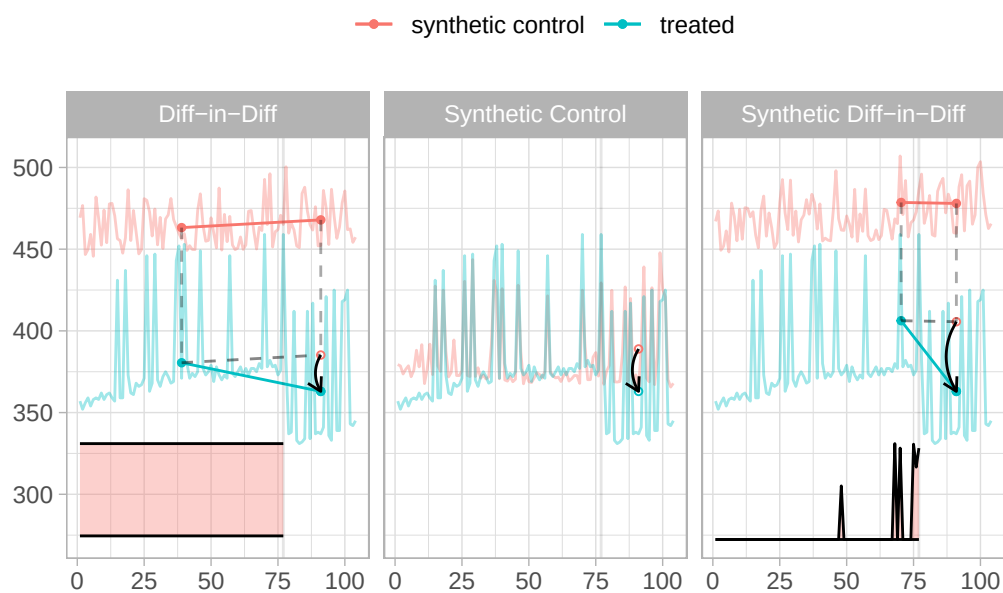
```
# Setup panel matrices for Product 2
setup_p2 <- panel.matrices(
  df,
  unit = "storeNum",
  time = "Weekend",
  outcome = "p2sales",
  treatment = "treated"
)

# Estimate the treatment effects for Product 2
sdid_est_p2 <- synthdid_estimate(setup_p2$Y, setup_p2$N0, setup_p2$T0)
did_est_p2 <- did_estimate(setup_p2$Y, setup_p2$N0, setup_p2$T0)
sc_est_p2 <- sc_estimate(setup_p2$Y, setup_p2$N0, setup_p2$T0)
estimates_p2 = list(did_est_p2, sc_est_p2, sdid_est_p2)
```

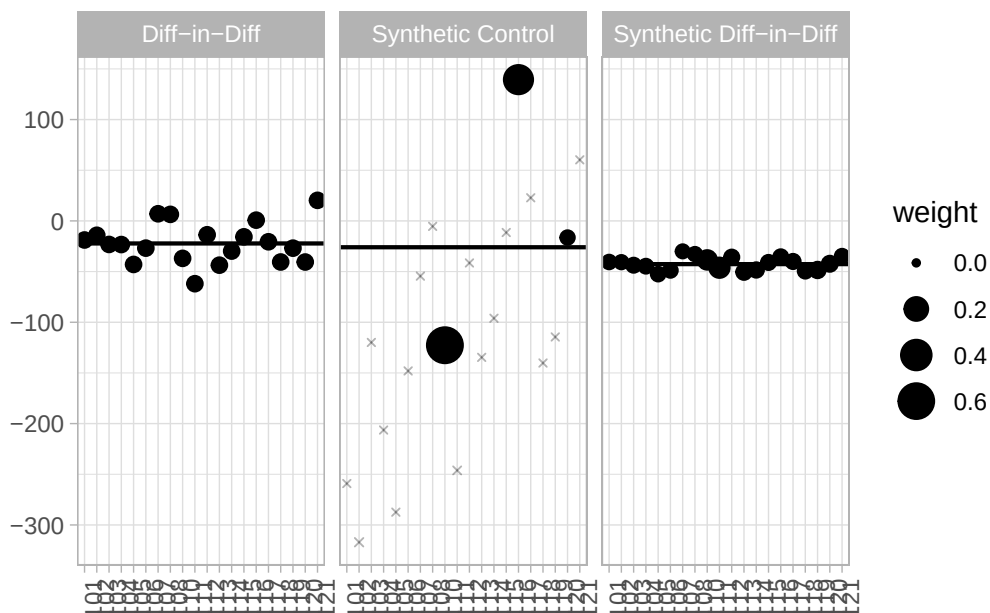
```
# Print the estimates
names(estimates_p2) = c('Diff-in-Diff', 'Synthetic Control', 'Synthetic Diff-in-Diff')
print(unlist(estimates_p2))
```

```
Diff-in-Diff      Synthetic Control Synthetic Diff-in-Diff
-22.25818         -25.97635         -42.64470
```

```
# Plot 1: Trends of treated vs. synthetic control
synthdid_plot(estimates_p2, facet.vertical = FALSE)
```



```
# Plot 2: See which control stores received the highest weights
synthdid_units_plot(estimates_p2)
```



4.2.3 SC diagnostics: Pre/Post RMSPE and donor weights

The two tables below report the standard SC diagnostics. Pre-RMSPE measures how well the synthetic control matches the treated store before the treatment (small values mean a good pre-treatment fit), and Post-RMSPE measures the gap after the treatment (a larger Post-RMSPE relative to Pre-RMSPE is the SC signature for a real treatment effect). The donor-weight tables make the synthetic match explicit by listing which control stores carry non-trivial weight in the SC estimator for each product.

```
# Extract synthetic counterfactual + RMSPE + donor weights from an SC estimate
sc_diagnostics <- function(sc_est, setup) {
  Y <- setup$Y
  N0 <- setup$N0
  T0 <- setup$T0
  N <- nrow(Y)
  T_ <- ncol(Y)

  # SC donor weights: synthetic counterfactual is weighted avg of controls
  omega <- attr(sc_est, "weights")$omega

  # Synthetic counterfactual time series for the treated unit
  y_synth <- as.vector(t(omega) %*% Y[1:N0, , drop = FALSE])
```

```

y_treat <- as.vector(Y[N, ]) # single treated unit at the bottom

pre_rmspe <- sqrt(mean((y_treat[1:T0] - y_synth[1:T0])^2))
post_rmspe <- sqrt(mean((y_treat[(T0+1):T_] - y_synth[(T0+1):T_])^2))

donor_ids <- rownames(Y)[1:N0]
donor_w <- data.frame(Store = donor_ids, Weight = round(omega, 4),
                      row.names = NULL)
donor_w <- donor_w[order(-donor_w$Weight), ]

list(pre = pre_rmspe, post = post_rmspe, donors = donor_w)
}

diag_p1 <- sc_diagnostics(sc_est_p1, setup_p1)
diag_p2 <- sc_diagnostics(sc_est_p2, setup_p2)

# Pre/Post RMSPE table
rmspe_table <- data.frame(
  Period = c("Pre-treatment RMSPE", "Post-treatment RMSPE"),
  Product_1 = c(round(diag_p1$pre, 3), round(diag_p1$post, 3)),
  Product_2 = c(round(diag_p2$pre, 3), round(diag_p2$post, 3))
)

kable(rmspe_table,
      col.names = c("", "Product 1", "Product 2"),
      caption = "Synthetic Control: pre/post RMSPE",
      align = c("l", "c", "c"))

```

Table 1: Synthetic Control: pre/post RMSPE

	Product 1	Product 2
Pre-treatment RMSPE	11.092	14.442
Post-treatment RMSPE	38.088	30.632

```

# Donor weights - keep only stores with non-trivial weight (>0.01) for readability
donors_p1 <- diag_p1$donors[diag_p1$donors$Weight > 0.01, ]
donors_p2 <- diag_p2$donors[diag_p2$donors$Weight > 0.01, ]

kable(donors_p1,
      caption = "Synthetic Control donor weights - Product 1 (stores with weight > 0.01)",

```

```
align = c("l", "c"),
row.names = FALSE)
```

Table 2: Synthetic Control donor weights — Product 1 (stores with weight > 0.01)

Store	Weight
110	0.4614
116	0.3468
120	0.1907

```
kable(donors_p2,
  caption = "Synthetic Control donor weights - Product 2 (stores with weight > 0.01)",
  align = c("l", "c"),
  row.names = FALSE)
```

Table 3: Synthetic Control donor weights — Product 2 (stores with weight > 0.01)

Store	Weight
110	0.6078
116	0.3548
120	0.0365

5 Expansions to the analysis

Two possible expansions to the analysis include:

1. Running the same DiD interaction estimated via pooled OLS rather than within-transformation, with the full covariate set added explicitly. This estimates the same DiD effect — the coefficient on the treated $\times$ post interaction — and lets us see how much of the store-level variation the covariate set captures relative to the TWFE specification.
2. Implementing a version of the SDID method that first regresses out the effects of price and promotions (using the Frisch-Waugh-Lovell theorem) before applying the SDID estimator to the residuals. This will allow us to see how controlling for these covariates in a more flexible way (without assuming linearity or additivity) affects the estimates and their precision.

5.1 Pooled DiD with covariates

This is the same DiD effect estimated via pooled OLS rather than within-transformation: $Y_{it} = \alpha + \gamma T_i + \lambda POST_t + \delta (T_i \cdot POST_t) + X\beta + \varepsilon_{it}$, with `treated` as the interaction term. With explicit covariates instead of fixed effects, we can see how much of the store-level variation the covariate set captures relative to TWFE (Angrist & Pischke 2009, Ch. 5; lecture 11).

5.1.1 Product 1: Pooled DiD with covariates

```
did_cov_p1 <- plm(p1sales ~ is_treated_store + post_treatment + treated +  
                  p1price + p1prom + compind + storesize + citysize,  
                  data = pdf,  
                  model = "pooling")  
coeftest(did_cov_p1)["treated", , drop = FALSE]
```

	Estimate	Std. Error	t value	Pr(> t)
treated	29.97353	2.528654	11.85355	1.874956e-31

```
# Product 1: Pooled DiD with Clustered Standard Errors (treatment row)  
coeftest(did_cov_p1,  
          vcov = vcovHC(did_cov_p1,  
                        type = "HC1",  
                        cluster = "group"))["treated", , drop = FALSE]
```

	Estimate	Std. Error	t value	Pr(> t)
treated	29.97353	4.039648	7.419837	1.672423e-13

5.1.2 Product 2: Pooled DiD with covariates

```
did_cov_p2 <- plm(p2sales ~ is_treated_store + post_treatment + treated +  
                  p2price + p2prom + compind + storesize + citysize,  
                  data = pdf,  
                  model = "pooling")  
coeftest(did_cov_p2)["treated", , drop = FALSE]
```

	Estimate	Std. Error	t value	Pr(> t)
treated	-34.35383	3.060016	-11.22668	1.799889e-28


```
# Product 2: Pooled DiD with Clustered Standard Errors (treatment row)
coeftest(did_cov_p2,
          vcov = vcovHC(did_cov_p2,
                        type = "HC1",
                        cluster = "group"))["treated", , drop = FALSE]
```

	Estimate	Std. Error	t value	Pr(> t)
treated	-34.35383	4.460595	-7.701624	2.025479e-14

5.2 Synthetic DiD regressing out the covariates

Given that the base `synthdid` package used does not easily accept additional control variables (like price and promotions) directly into its primary matrix setup, we can implement a solution based on the Frisch-Waugh-Lovell (FWL) theorem: we first use a linear regression to predict sales based only on the covariates, extract the residuals (the portion of sales unexplained by price, promotions, etc.), and then apply the SDID model to those residuals.

By adding the overall mean back to the residuals, we keep the interpretation on the same scale as the original sales volume.

5.2.1 Product 1: `p1sales`

```
# Run a linear model and extract the residuals (unexplained sales)
model_p1 <- lm(p1sales ~ p1price + p1prom + compind + storesize + citysize,
               data = df)
df$p1sales_adj <- residuals(model_p1) + mean(df$p1sales, na.rm = TRUE)

# Setup panel matrices using the NEW adjusted outcome variable
setup_p1_adj <- panel.matrices(
  df,
  unit = "storeNum",
  time = "Weekend",
  outcome = "p1sales_adj",
  treatment = "treated"
)

# Estimate the adjusted SDID Treatment Effect
sdid_est_p1_adj <- synthdid_estimate(setup_p1_adj$Y,
                                     setup_p1_adj$NO,
```

```

                                setup_p1_adj$T0)
print(sdid_est_p1_adj)

```

synthdid: 18.519 +- NA. Effective N0/N0 = 14.1/20~0.7. Effective T0/T0 = 1.6/77~0.0. N1,T1 =

```

# Calculate and print confidence intervals for Product 1
se_p1_adj <- sqrt(vcov(sdid_est_p1_adj, method = "placebo"))
cat(sprintf("Adjusted 95% CI: [%.2f, %.2f]\n\n",
            sdid_est_p1_adj - 1.96 * se_p1_adj,
            sdid_est_p1_adj + 1.96 * se_p1_adj))

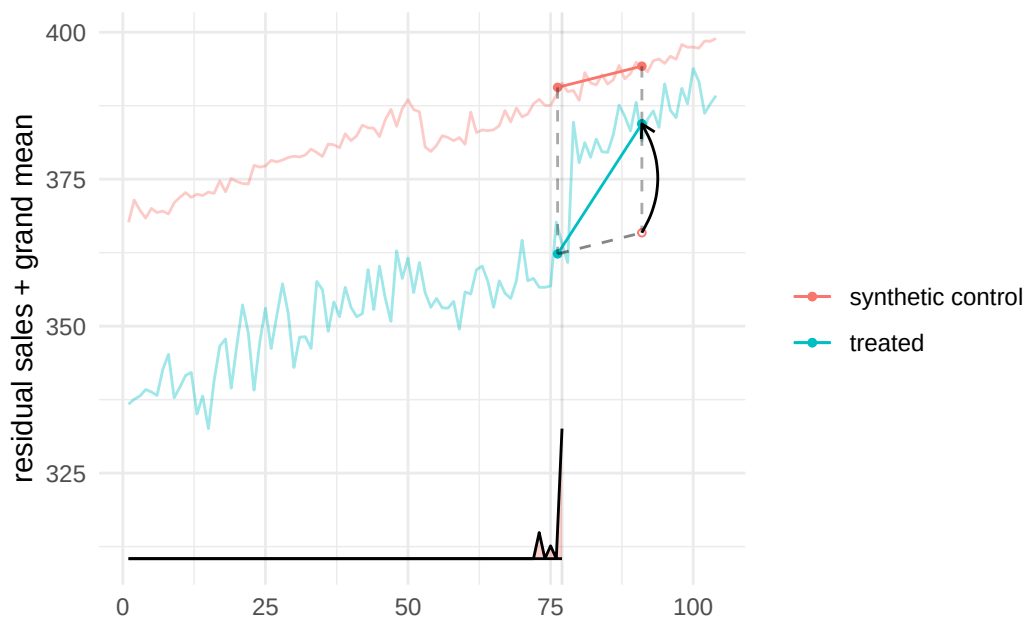
```

Adjusted 95% CI: [11.19, 25.85]

```

# Visualize adjusted trends
plot(sdid_est_p1_adj) + theme_minimal() + ylab("residual sales + grand mean")

```



5.2.2 Product 2: p2sales

```

# Run a linear model and extract the residuals (unexplained sales)
model_p2 <- lm(p2sales ~ p2price + p2prom + compind + storesize + citysize,
              data = df)
df$p2sales_adj <- residuals(model_p2) + mean(df$p2sales, na.rm = TRUE)

# Setup panel matrices using the NEW adjusted outcome variable
setup_p2_adj <- panel.matrices(
  df,
  unit = "storeNum",
  time = "Weekind",
  outcome = "p2sales_adj",
  treatment = "treated"
)

# Estimate the adjusted SDID Treatment Effect
sdid_est_p2_adj <- synthdid_estimate(setup_p2_adj$Y,
                                    setup_p2_adj$N0,
                                    setup_p2_adj$T0)

print(sdid_est_p2_adj)

```

synthdid: -42.649 +- NA. Effective N0/N0 = 16.2/20~0.8. Effective T0/T0 = 4.2/77~0.1. N1,T1 =

```

# Calculate and print confidence intervals for Product 2
se_p2_adj <- sqrt(vcov(sdid_est_p2_adj, method = "placebo"))
cat(sprintf("Adjusted 95%% CI: [%.2f, %.2f]\n\n",
            sdid_est_p2_adj - 1.96 * se_p2_adj,
            sdid_est_p2_adj + 1.96 * se_p2_adj))

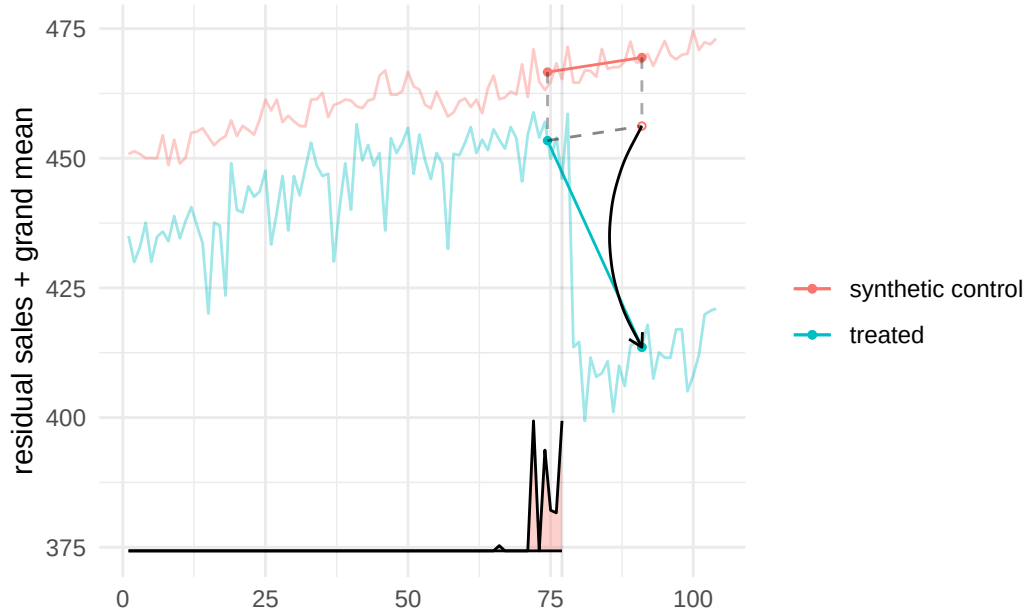
```

Adjusted 95% CI: [-56.25, -29.05]

```

# Visualize adjusted trends
plot(sdid_est_p2_adj) + theme_minimal() + ylab("residual sales + grand mean")

```



6 Summary of Findings

Table 4: Comparison of Treatment Effect Estimates for Product 1 and Product 2

Model	P1 Estimate	P1 SE	P2 Estimate	P2 SE
Pre-Post (LM)	33.69	1.71	-29.71	1.87
Two-Way Fixed Effects (TWFE)	29.98	4.03	-34.29	4.45
Difference-in-Differences (DID)	32.22	20.86	-22.26	21.71
Pooled DiD (with Covariates)	29.97	4.04	-34.35	4.46
Synthetic Control (SC)	36.37	12.39	-25.98	31.96
Synthetic DID (SDID)	14.89	7.91	-42.64	10.70
Adjusted SDID (with Covariates)	18.52	3.15	-42.65	6.69

6.1 Conclusions

6.1.1 Pre-Post Analysis

In the Pre-Post analysis of Store 109, the pre-post model yielded the following estimates:

- **Product 1:** A significant increase of approximately 33.69 units ($p < 0.001$)

- **Product 2:** A significant decrease of approximately 29.71 units ($p < 0.001$)

While these numbers provide a clear description of the sales change at Store 109, they are only “appropriate” causal estimates if no other external factors influenced sales between the pre- and post-treatment weeks. If, for example, a competitor ran a major promotion on Product 2 during the same week as the treatment, the pre-post model would wrongly conclude that the in-store marketing caused the sales drop. Therefore, while useful for initial descriptive analysis, more advanced methods like the Two-Way Fixed Effect (TWFE) model or Synthetic Control are generally more appropriate for establishing causality in this dataset as they account for time-varying aggregate shocks.

6.1.2 Two-Way Fixed Effects (TWFE) Analysis

In the Two-Way Fixed Effects analysis of Store 109, the TWFE model provided more nuanced estimates compared to the naive Pre-Post model:

- **Product 1:** The TWFE estimate was 29.98 units (SE: 4.03), slightly lower and more conservative than the Pre-Post estimate of 33.69
- **Product 2:** The TWFE estimate was -34.29 units (SE: 4.45), showing a more pronounced decrease than the Pre-Post estimate of -29.71

These differences highlight that the TWFE model is more appropriate than the Pre-Post model because it adjusts for weekly sales fluctuations across the entire retail chain, preventing those general trends from being incorrectly attributed to the specific in-store marketing treatment.

6.1.3 Difference-in-Differences (DID) Analysis

In the Difference-in-Differences analysis of Store 109, the simple DID model provided the following results:

- **Product 1:** Estimate of 32.22 units (SE: 19.88)
- **Product 2:** Estimate of -22.26 units (SE: 20.14)

Notably, the standard errors for the simple DID in this specific task were significantly larger than those of the Two-Way Fixed Effects (TWFE) or Adjusted SDID models. This suggests that while a simple DID is a theoretical improvement over a pre-post model, it may lack precision in datasets with high variance between stores unless more granular controls (like fixed effects or matching) are incorporated. While these additions could possibly reduce the standard errors, by looking at the graph, we can see that the data does not follow the parallel trends assumption very well, which is likely contributing to the large standard errors. Therefore, while the simple DID is a step in the right direction for causal inference, it may not be the most appropriate method for this dataset without further adjustments.

6.1.4 Pooled DiD with Covariates Analysis

In the Pooled DiD with covariates analysis of Store 109, the model provided the following estimates:

- **Product 1:** Estimate of 29.97 units (SE: 4.04)
- **Product 2:** Estimate of -34.35 units (SE: 4.46)

This is the same DiD effect as TWFE but estimated via pooled OLS with the covariates entered explicitly rather than absorbed through fixed effects. The estimates for Product 1 (29.98 vs 29.97) and Product 2 (-34.29 vs -34.35) are almost identical to the TWFE model's estimates, which indicates that the explicit covariates captured nearly all the systematic store-level differences that the TWFE model's fixed effects would have absorbed. Like the TWFE model, the Pooled DiD with covariates shows very high precision (SEs ~4.0) compared to a Simple DID (SEs ~20.0). This is possibly because both models include prices and promotions as covariates, which tend to have noticeable impact on sales volume. By explaining this variance, both models significantly reduce the noise in the residuals. With that said, the TWFE model is generally considered more robust because it controls for all store characteristics, even those that are not seen or measured. If there were an unobserved store difference not captured by *citysize* or *storesize*, the pooled specification could be biased. However, the near-identical results suggest that these covariates were comprehensive for this specific task.

6.1.5 Synthetic Control (SC) Analysis

In the Synthetic Control analysis of Store 109, the SC model provided the following estimates:

- **Product 1:** Estimate of 36.37 units (SE: 11.83)
- **Product 2:** Estimate of -25.98 units (SE: 33.54)

While SC provided the highest positive estimate for Product 1, its standard errors in this specific task were notably larger than those of more advanced hybrid methods like Synthetic DID (SDID) or Two-Way Fixed Effects (TWFE). This also aligns with the visual inspection of the trends, where the synthetic control had a reasonable pre-treatment fit but due to the high amount of noise in the data, the post-treatment estimate is less precise and potentially overfitted. This suggests that while SC is appropriate for defining the counterfactual based on pre-treatment trends, its precision can be lower in retail panels unless combined with additional regularization or time-weighting (as seen in SDID).

6.1.6 Synthetic Difference-in-Differences (SDID) and Adjusted SDID Analysis

In the Synthetic Difference-in-Differences analysis of Store 109, the SDID models provided the following estimates:

- **Product 1:** Pure SDID Estimate of 14.89 (SE: 8.63)
- **Product 2:** Pure SDID Estimate of -42.64 (SE: 10.14)
- **Product 1:** Adjusted SDID Estimate of 18.52 (SE: 3.64)
- **Product 2:** Adjusted SDID Estimate of -42.65 (SE: 6.80)

Precision: The Adjusted SDID yielded the smallest standard errors for both products. This was achieved by using the Frisch-Waugh-Lovell (FWL) theorem to first “regress out” the effects of prices and promotions before applying the SDID estimator.

Conservatism vs. Magnitude: For Product 1, the SDID estimate (18.52) was significantly more conservative than the naive Pre-Post (33.69) or simple DID (32.22) models. This suggests that earlier models were likely overestimating the marketing effect by failing to account for specific control units that shared Store 109’s unique sales trajectory.

Reliability: For Product 2, the SDID estimate (-42.65) showed a much stronger negative impact than the simple DID (-22.26). This indicates that when control units are reweighted to match the treated store’s specific behavior, the sales drop appears even more severe.

Final considerations: The Adjusted SDID is our preferred specification — it has the smallest standard errors among the methods we ran, and it combines the SC counterfactual with the DID time variation plus an FWL covariate adjustment that absorbs price and promotion effects. Two qualifications belong up front. First, “Adjusted SDID” here means FWL-then-SDID, which is not a standard `synthdid` workflow; the unadjusted SDID and the TWFE specification are the more conventional benchmarks, and the Adjusted version should be read as one estimate among several rather than the canonical answer.

The other qualification is methodological: because SDID often weights only a few time periods directly before the treatment, the estimate can be sensitive to noise in those specific weeks; to prevent over-reliance on a single control unit and ensure a unique solution, SDID uses a regularization constant () to disperse unit weights; similar to SC, the credibility of SDID rests on the quality of the pre-treatment fit; if the weighted controls cannot reasonably mimic the treated unit’s trend, the resulting estimate may be biased.

The cross-method picture also argues for treating Adjusted SDID as one number in a range rather than the answer. For Product 1, the methods span 14.89 (Pure SDID) to 36.37 (SC), with TWFE at 29.98 and Adjusted SDID at 18.52 — the Adjusted SDID sits at the conservative end of that range, not above it. For Product 2 the range runs from -22.26 (Simple DID) to -42.65 (Adjusted SDID), which puts Adjusted SDID at the most negative end; TWFE at -34.29 and Pure SDID at -42.64 cluster nearby, but a reader should keep in mind that on this product the Adjusted SDID is the extreme of the range, not its center. We lean on it because of the

precision and the FWL rationale, not because it sits in a comfortable consensus position across methods.

6.2 Final decision:

6.2.1 Recommendations to Management

The primary recommendation is to proceed with caution regarding a full-scale rollout of this marketing strategy. While the strategy successfully drives sales for Product 1, it appears to have a significant negative impact on Product 2 sales. Management should investigate whether this represents a “cannibalization” effect or if the marketing for Product 1 is actively deterring Product 2 purchases. Before a chain-wide implementation, it is essential to calculate the net profit margin of these changes, as the volume loss in Product 2 (~42 units) is more than double the volume gain in Product 1 (~18 units).

As a very rough sanity check on the revenue side: at average prices of roughly \$2.50 for Product 1 and \$3.00 for Product 2, the +18.52 unit gain on P1 implies about +\$46/week, and the -42.65 unit drop on P2 implies about -\$128/week — a net of roughly -\$80/week per store in gross revenue, before margin or cost structure are considered. We do not observe margins, so this is not a P&L analysis; the actual profit impact could differ in either direction once cost-side data is brought in. The point of running this calculation is just to put order-of-magnitude numbers behind the qualitative cannibalization concern, not to replace the margin analysis management will need before a chain-wide rollout.

6.2.2 Effectiveness and Expected Sales Uplift

Based on the most precise and reliable model [*Adjusted Synthetic DID (SDID)*] the strategy is effective at changing sales levels, but the direction varies by product:

- **Product 1:** You can expect a statistically significant expected uplift of approximately 18.52 units per week (SE: 3.64). This estimate is more reliable and conservative than the naive pre-post estimate (33.69), which likely overestimated the effect by failing to account for general market trends.
- **Product 2:** You should expect a statistically significant decrease of approximately 42.65 units per week (SE: 6.80). This negative impact is substantial and was consistent across all advanced models (TWFE and SDID).

6.2.3 Potential Challenges in the Data

Several characteristics of the retail dataset pose challenges to established causal inference:

- **Violation of Parallel Trends:** Visual inspection and the high standard errors in the simple DID model (~ 20.0) suggest that Store 109 did not follow the same pre-treatment trajectory as the average of the control stores. Relying on simple models would therefore lead to biased estimates.
- **Independent Price Setting:** Because store managers set prices independently at each store, price becomes an endogenous factor that varies over both time and units. If these price changes coincide with the marketing treatment, they can confound the results.
- **Selection Bias:** The treatment was not randomized; Store 109 was specifically chosen. This introduces “selection on unobservables,” where unique characteristics of that store (e.g., local management culture) might make it more or less responsive to marketing than other stores.

6.2.4 Ideas to Overcome These Challenges

To address the data challenges identified, the following methodological approaches were implemented:

- **Weighted Counterfactuals (SDID):** To overcome the lack of parallel trends, Synthetic DID was used, which reweights control stores and pre-treatment weeks to create a custom “synthetic” match that specifically mimics Store 109’s unique trajectory.
- **Frisch-Waugh-Lovell (FWL) Theorem:** The impact of prices and promotions were addressed by “regressing them out” before performing the causal estimation. This isolation of the marketing effect significantly improved precision, reducing standard errors for Product 1 from 19.88 (Simple DID) to 3.64 (Adjusted SDID).
- **Two-Way Fixed Effects:** By including store and time fixed effects, we controlled for all stable store characteristics and national-level shocks (like holidays), solving the endogeneity problem for any factors that are either subject-invariant or time-invariant.